PYTHON RECURSION

Deep handwritten-style DSA notes | Beginner to interview level

1. What is recursion?

Recursion means a function calls itself to solve a smaller version of the same problem. Every recursive solution needs (1) a base case to stop and (2) a recursive case that moves toward the base case.

```
def countdown(n):
    if n == 0:       # base case
        print("Done")
        return
    print(n)
    countdown(n - 1) # smaller problem

countdown(3)  # 3, 2, 1, Done
```

2. How the call stack works

Each function call gets its own stack frame (local variables and a return location). Calls stack up; when a base case returns, frames finish in reverse order.

```
countdown(3)
  countdown(2)
    countdown(1)
      countdown(0) -> Done
    return from countdown(1)
  return from countdown(2)
return from countdown(3)
```

Key insight: Going down = building calls. Coming back = unwinding the stack. The maximum number of simultaneously active calls determines recursion-stack space.

3. Base case versus recursive case

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be nonnegative")
    if n <= 1:         # 0! = 1! = 1
        return 1
    return n * factorial(n - 1)
```

*For factorial(4): 4 * factorial(3) -> 4 * 3 * factorial(2) -> 4 * 3 * 2 * factorial(1) -> 24. Time O(n), auxiliary space O(n) due to the stack.*

4. Trace a recursion tree by hand

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

fib(4)
        fib(4)
       /      \
    fib(3)   fib(2)
    /   \    /   \
 fib(2) fib(1) fib(1) fib(0)
 /   \
fib(1) fib(0)
```

Naive Fibonacci repeats the same subproblems. Its time is O(2^n) as a loose upper bound (more precisely exponential near 1.618^n), while stack space is O(n).

5. Recursion versus iteration

```
# Iterative factorial: O(n) time, O(1) extra space
def factorial_loop(n):
    if n < 0: raise ValueError("n must be nonnegative")
    ans = 1
    for x in range(2, n + 1):
        ans *= x
    return ans
```

Loops often use less memory in Python. Recursion can express tree traversal, divide-and-conquer and backtracking more naturally. Python has a recursion depth limit and does not optimize tail recursion away.

6. Recursion patterns

Linear recursion: one recursive call (factorial, sum). Branching recursion: multiple calls (naive Fibonacci, subset generation). Divide and conquer: split into smaller subproblems, then combine (merge sort, binary search). Backtracking: choose, recurse, undo choice (subsets, permutations).

7. Complexity: derive it, do not guess

For one call reducing n by 1 with O(1) work: T(n)=T(n-1)+O(1), so time O(n). For two calls on n-1: T(n)=2T(n-1)+O(1), exponential. For binary search: T(n)=T(n/2)+O(1), time O(log n).

Space is measured by maximum active recursion depth times memory per frame, plus any additional arrays, lists, or copied slices.

8. Sum of array using recursion

```
def array_sum(arr, i=0):
    if i == len(arr):
        return 0
    return arr[i] + array_sum(arr, i + 1)

print(array_sum([2, 4, 6]))  # 12
```

Trace: sum(0)=2+sum(1)=2+4+sum(2)=2+4+6+sum(3)=12. Time O(n), auxiliary stack O(n). Avoid arr[1:] slices here: copying slices increases work and memory.

9. Reverse a string

```
def reverse(s):
    if len(s) <= 1:
        return s
    return reverse(s[1:]) + s[0]

print(reverse("python"))  # nohtyp
```

This is easy to understand but slicing and repeated string concatenation can make the total time O(n^2), with O(n^2) peak memory possible across copied slices/strings. For practical Python, s[::-1] is simpler and typically O(n) time and O(n) space.

10. Recursive binary search

```
def binary_search(a, target, left, right):
    if left > right:
        return -1
    mid = (left + right) // 2
    if a[mid] == target:
        return mid
    if a[mid] < target:
        return binary_search(a, target, mid + 1, right)
    return binary_search(a, target, left, mid - 1)

a = [1, 3, 5, 7, 9]
print(binary_search(a, 7, 0, len(a)-1)) # 3
```

Requires a sorted array. At every level, search space halves. Time O(log n), auxiliary stack O(log n).

11. Fast Fibonacci using memoization

```
from functools import lru_cache

@lru_cache(None)
def fib_fast(n):
    if n <= 1: return n
    return fib_fast(n - 1) + fib_fast(n - 2)
```

Memoization stores answers to repeated subproblems. Counting arithmetic operations under the usual fixed-cost model: O(n) time and O(n) cache plus O(n) stack space. For very large n, integer arithmetic and recursion limits also matter.

12. Backtracking: generate all subsets

```
def subsets(nums):
    result = []
    path = []
    def dfs(i):
        if i == len(nums):
            result.append(path.copy())
            return
        dfs(i + 1)              # skip
        path.append(nums[i])    # choose
        dfs(i + 1)              # explore
        path.pop()              # undo
    dfs(0)
    return result

print(subsets([1, 2]))
# [[], [2], [1], [1, 2]]
```

*There are 2^n subsets. Creating output lists takes O(n * 2^n) total time and output storage. Excluding the returned output, path + recursion stack use O(n) auxiliary space.*

13. Common mistakes and debugging

1. Missing base case -> infinite recursion until RecursionError. 2. Recursive call does not reduce the problem. 3. Forgetting return -> None. 4. Modifying shared mutable lists without undoing changes. 5. Confusing total number of calls with maximum stack depth. 6. Assuming recursion is always faster than loops.

```
# BUG: no return before recursive result
def bad_sum(n):
    if n == 0: return 0
    n + bad_sum(n - 1)  # missing return!

# FIX
def good_sum(n):
    if n == 0: return 0
    return n + good_sum(n - 1)
```

14. Five-step method for any recursion question

Step 1: Define what f(state) means. Step 2: Write the smallest stopping case. Step 3: Identify the smaller state. Step 4: Combine or return the smaller answer. Step 5: Trace a tiny input and calculate time and stack space.

15. Practice exercises (solve before checking)

Easy: (1) Print 1 to n recursively. (2) Print n to 1. (3) Sum 1..n. (4) Factorial. (5) Count digits. (6) Power(a,b) for b >= 0. (7) Check palindrome. (8) Find max in list.

Intermediate: (9) Binary search recursively. (10) Fibonacci with memoization. (11) Generate subsets. (12) Generate valid parentheses. (13) Reverse a linked list recursively. (14) Tree preorder traversal.

16. Answer hints + target complexities

(1)-(4): one call per integer -> O(n) time, O(n) stack. (5): divide number by 10 each call -> O(d) time and stack, where d is digit count. (6): repeated multiplication O(b) or exponentiation by squaring O(log b). (7): use left/right indices -> O(n) time, O(n) stack. (8): compare element with max of rest -> O(n) time, O(n) stack.

*(9): O(log n) time and stack. (10): O(n) arithmetic steps and O(n) cache/stack under fixed-cost arithmetic. (11): O(n*2^n) time including output. (12): output-sensitive exponential search. (13): O(n) time and stack. (14): O(n) time, O(h) stack where h is tree height.*

17. Mini self-test

Q: What happens if base case is never reached? A: Calls continue until Python raises RecursionError (or another resource limit is reached).

Q: If recursion makes n calls, is space always O(n)? A: No. Space depends on maximum simultaneously active calls and allocations, not simply total calls.

Q: Why is naive Fibonacci slow? A: It recomputes overlapping subproblems. Memoization avoids that repetition.

Q: Does tail recursion use O(1) stack in Python? A: No, Python does not guarantee tail-call optimization.

18. Interview-ready summary

A recursive function solves a problem by calling itself on smaller inputs until it reaches a base case. I check that the input gets closer to the base case, trace the call stack, and derive time complexity from the number of calls and work per call. Auxiliary space comes from the maximum recursion depth and any extra data structures.

Keep practicing: understand -> trace -> code -> analyze -> optimize.